CYBR 437 Secure Coding

Lab 3 - PwnDBG

Name: Artem Peychev
Assignment: Lab 3
Due date: October 14, 2026

Part 1

Question 1

What is the name of the C file that contains main?

The name of the C file containing he main function is called cybr437lab2main.c

Commands:

```
info functions main
```

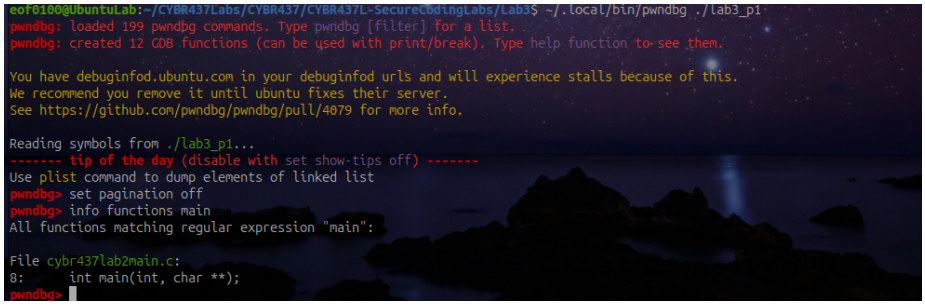


Figure 1: Part 1, Question 1

Question 2

Check the security of the binary and display the output inside pwndbg.

I used checksec inside pwndbg and the binary has Full RELRO, a stack canary, NX, and PIE enabled. SHSTK and IBT are also marked enabled. The binary is not stripped and does contain debugging information.

Commands:

```
checksec
```

Question 3

Disassemble main and identify the user-defined functions (print statements and function calls in proper order).

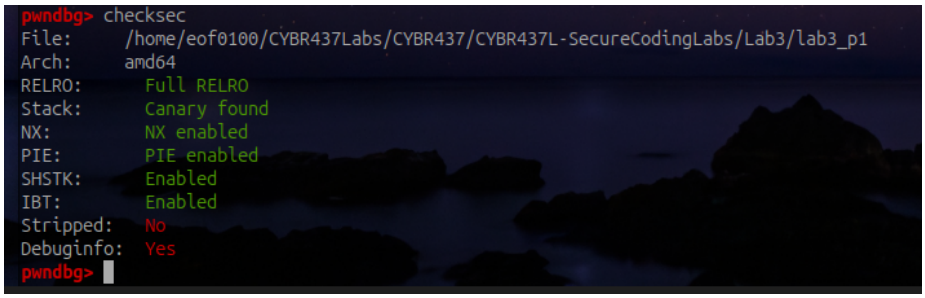


Figure 2: Part 1, Question 2

The user-defined functions are <simpleCounter> and <simpleIf>. The calls are in the following order: puts, simpleCounter(3), printf, scanf, simpleIf(num), and simpleCounter(num).

Commands:

```
set disassembly-flavor intel
disassemble main
```

Question 4

Set a breakpoint at main and on each of the identified functions. Run the program.

Commands:

```
break main
break simpleCounter
break simpleIf
run
set $labbase = (unsigned long)&main - 0x12a3
```

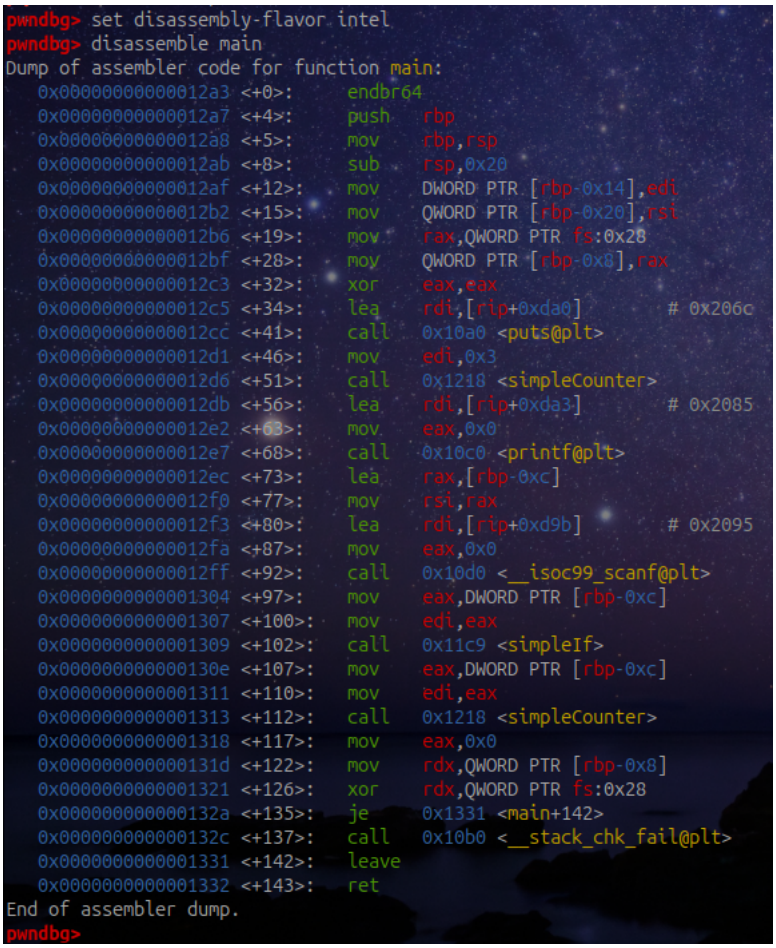


Figure 3: Part 1, Question 3



Question 5

What line are you at? What are the contents of the first string in main?

The program is stopped at line 9 in cybr437lab2main.c. The first string in main is:

"This is a pwndbg program".

Commands:

```
info line *$pc
x/s $labbase+0x206c
```

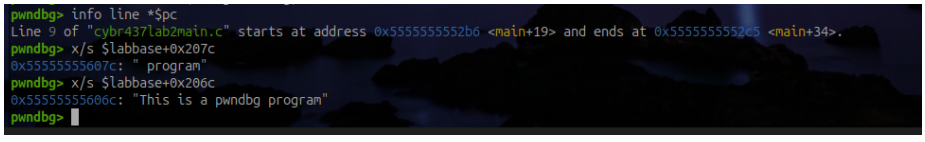


Figure 4: Part 1, Question 5

Question 6

Display the current breakpoints, delete the one for main and set a new one at line 23, show the updated list of breakpoints.

I deleted breakpoint 1 at main and added a breakpoint at line 23 in cybr437lab2main.c but the breakpoints of simpleCounter and simpleIf are still there.

Commands:

```
info breakpoints
delete 1
break cybr437lab2main.c:23
info breakpoints
```

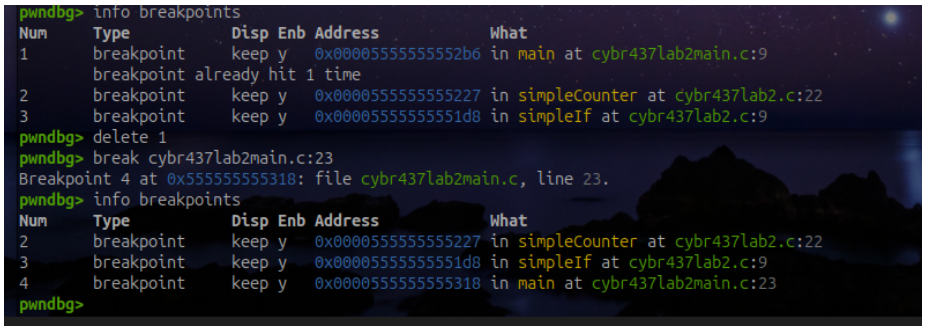


Figure 5: Part 1, Question 6

Question 7

Continue to the next breakpoint. What is the name and the value being passed into the function called?

The function called is simpleCounter, the parameter is named num and its value is 0x03 which is 3 in hexadecimal notation.

Commands:

```
continue
info args
```



Question 8

How do you show the current function and the one that called it? Show output.

I used backtrace, frame #0 shows simpleCounter as the current function with frame #1 is the caller.

Commands:

`backtrace`

Question 9

Disassemble this function you are in, how many print functions are in this function? Hint: There is more than one type of print statement.

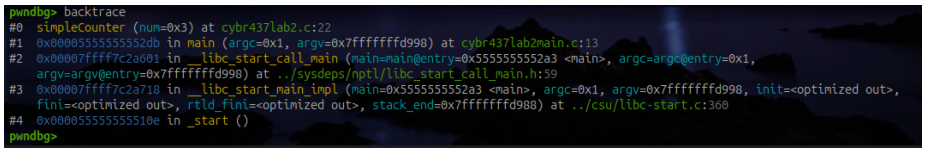


Figure 6: Part 1, Question 8

There are four output function calls in the disassembled code: 3 calls to printf and 1 call to putchar.

Commands:

`disassemble simpleCounter`



Figure 7: Part 1, Question 9

Question 10

Examining the disassembled code, grab the memory address of the constant from the function you're stopped at and display its location in the process image.

The constant VAR has a vlue of 100 at address 0x555555556068 which is stored real only data section called .rodata.

Commands:

```
p/x &VAR
p VAR
x/wd $labbase+0x2068
vmmap $labbase+0x2068
info files
```

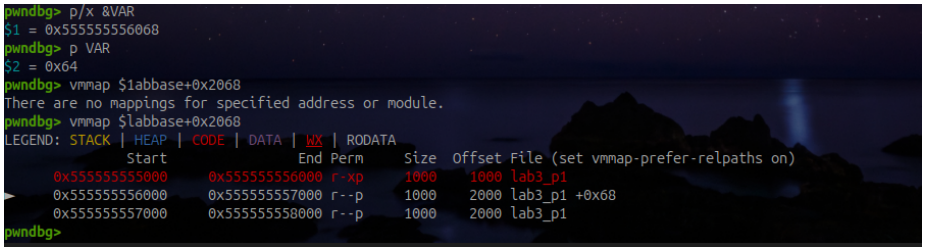


Figure 8: Part 1, Question 10

Question 11

Continue running (enter 15). What function are you currently stopped at, what is the name and the value being passed into the function you are currently stopped at?

The program stopped in simpleIf with parameter named num and the value passed in is 15.

Commands:

```
continue
# When the program asks Enter a number, type 15 and press Enter.
info args
backtrace
```



Question 12

What is the value of the first string in this function you are stopped at, and what is the name and the type of the global variable in the function where you are stopped in? Hint: notice the legend.

The first string in simpleIf says “Congrats you won”. The global var is MAX of type constant double.

Commands:

```
disassemble simpleIf
x/s $labbase+0x2008
x/s $labbase+0x2019
whatis MAX
p MAX
p/x &MAX
vmmap $labbase+0x4020
```



Figure 9: Part 1, Question 12

Question 13

Look for the cmp instruction in the assembly from the disassembled code. What value is the variable from question 14 being compared to?

The variable is compared to 0xc, which is 12 in decimal.

Commands:

```
disassemble simpleIf
p num
```

Question 14

What area of the process image is the variable s located, what storage class is it? Hint: notice the value of s as it enters the function that is called twice.



Figure 10: Part 1, Question 13

The variable s is stored in .data section and has a Statoc Storage Class. Its value starts at 1 and then it is 4 when simpleCounter function is called the second time.

Commands:

```
p s
p/x &s
vmmap $labbase+0x4010
info variables s
continue
info args
p s
p/x &s
vmmap $labbase+0x4010
```



Question 15

Display 64 addresses of the stack (NOTE: this is not backtrace).

I used x/64gx $rsp command to display 64 8-byte items on the stack starting at the current pointer in the stack.

Commands:

`x/64gx $rsp`



Figure 11: Part 1, Question 15

Part 2: Stack frames for main and foo

Using the addresses supplied in the handout, draw the stack frames for both `main` and `foo`. Clearly label each variable and its memory location. High memory must be at the top.

High Memory

Variable	Memory address	Location
array[5] = 60	0x7fffdb770234	stack
array[4] = 50	0x7fffdb770230	stack
array[3] = 40	0x7fffdb77022c	stack
array[2] = 30	0x7fffdb770228	stack
array[1] = 20	0x7fffdb770224	stack
array[0] = 10 (array base)	0x7fffdb770220	stack
int *ptr points to x	0x7fffdb770218	stack
char *word points to heap allocation	0x7fffdb770210	stack
int *ptr2 points to y	0x7fffdb770208	stack
double *dptr2 points to val2	0x7fffdb770200	stack
double val2 = 3.14159	0x7fffdb7701f8	stack
double *dptr points to val	0x7fffdb7701f0	stack
double val = 4.9567	0x7fffdb7701e8	stack
int y = 99	0x7fffdb7701e4	stack
int x = 12	0x7fffdb7701e0	stack
a[3] = 128	0x7fffdb7701ac	stack
a[2] = 64	0x7fffdb7701a8	stack
a[1] = 32	0x7fffdb7701a4	stack
a[0] = 16 (array base)	0x7fffdb7701a0	stack
int *b points to calloc allocation	0x7fffdb770198	stack
int *c points to malloc allocation	0x7fffdb770190	stack
double d = 512	0x7fffdb770188	stack
int i = 1024	0x7fffdb770184	stack
int x = 256	0x7fffdb770180	stack
b allocation: 32 ints, 128 bytes	0x55e8969576f0	heap
c allocation: 16 bytes	0x55e8969576d0	heap
word allocation: 10 chars, 10 bytes	0x55e8969572a0	heap

Low Memory

Figure 12: Stack frames and heap allocations from the supplied addresses